

A Big Decimal Type

Doc No: P2159R1

Date: 2022-12-24

Author: Bill Seymour

Reply to: stdbill.h@pobox.com

Audience: LEWG-I, SG6 (Numerics)

Contents

- [Introduction](#)
 - [Synopsis](#)
 - [Detailed Descriptions](#)
 - [The decimal Class](#)
 - [A Member Type and Some Constants](#)
 - [Special Member Functions and swap](#)
 - [Construction from Other Types](#)
 - [Assignment from Other Types](#)
 - [Conversion to Other Types](#)
 - [Rounding and Scale](#)
 - [Observers](#)
 - [Mutators](#)
 - [Comparisons](#)
 - [Arithmetic](#)
 - [Size, etc.](#)
 - [Non-member Functions](#)
 - [Miscellaneous Functions](#)
 - [Operators](#)
 - [<cmath>-like Functions](#)
-

Introduction

This is a revision of P2159R0 which was discussed in Kona; but the author wasn't able to attend. The author will be present at the face-to-face meeting in Issaquah.

This paper proposes an exact decimal type that can be used as either a fixed point type or a floating point type with an implementation-defined maximum precision. An instance will never be a negative zero.

Users can specify at run time a rounding mode and a *scale*, the number of decimal digits to the right of the decimal point.

Instances would normally act like a floating point type internally, and so addition, subtraction and multiplication would be exact and so could represent more than the implementation-defined maximum number of digits. Division, if it would generate a quotient of greater than the maximum precision, will generate a guard digit and then round to the maximum precision.

Rounding to the user's desired scale occurs when the value becomes externally visible (e.g., `to_integer()`, `to_string()`), or when the user explicitly calls one of the member `round()` overloads. Users can turn off rounding (except for division) by explicitly requesting an out-of-band value for the scale, thus making the object seem to be a floating point value even to the outside world.

Overloads of the few `<cmath>` functions that return exact values are provided as non-member functions.

This paper assumes that we have, from [P1889R1](#), the rounding enumeration (§3.3) and the unbounded integer class (§9.3).

Issues:

- Should all arithmetic operations round to the user's desired scale unless the user has explicitly requested a "floating" scale?
- Should the type support NaNs and/or infinities? (This paper assumes support for quiet NaNs and infinities.)
 - Should this depend of whether the C++ implementation's fundamental floating point types have them?

Synopsis

```
class decimal final
{
public:
    //
    // A member type and some constants:
    //
    using scale_type = int;
    static constexpr scale_type max_scale = implementation-defined;
    static constexpr scale_type floating_scale = INT_MIN;

    static constexpr int max_digits = implementation-defined;

    static constexpr rounding default_rounding = rounding::tie_to_even;
    static constexpr rounding bankers_rounding = rounding::tie_away_zero;

    static constexpr decimal inf = a value that represents positive infinity;
    static constexpr decimal nan = a value that represents a quiet NaN;

    //
    // Special member functions and swap:
    //
    explicit decimal(rounding = default_rounding, scale_type = 0);

    decimal(const decimal&);
    decimal(decimal&&);

    decimal& operator=(const decimal&);
    decimal& operator=(decimal&&);

    ~decimal() noexcept;

    void swap(decimal&);

    //
    // Construction from other types:
    //
    template<typename FundamentalArithmeticType>
        decimal(FundamentalArithmeticType, scale_type = see below, rounding = default_rounding);
    explicit decimal(const integer&, scale_type = 0, rounding = default_rounding);
    explicit decimal(const char*, rounding = default_rounding, const char** = nullptr);
    explicit decimal(const std::string&, rounding = default_rounding, std::size_t* = nullptr);

    //
    // Assignment from other types:
    //
    template<typename FundamentalArithmeticType>
        decimal& operator=(FundamentalArithmeticType);
    template<typename FundamentalArithmeticType>
        decimal& assign(FundamentalArithmeticType, scale_type = see below, rounding = default_rounding);
    decimal& assign(const integer&, scale_type = 0, rounding = default_rounding);
    decimal& assign(const char*, rounding = default_rounding, const char** = nullptr);
    decimal& assign(const std::string&, rounding = default_rounding, std::size_t* = nullptr);

    //
    // Conversion to other types:
    //
    explicit operator bool() const noexcept;
```

```

explicit operator long double() const noexcept;

integer to_integer(rounding = rounding::all_to_zero) const;

std::string to_string(bool scientific = false) const;
std::string to_string(int precision, bool scientific = false) const;

//
// Rounding and scale:
//
rounding get_rounding() const noexcept;
void set_rounding(rounding) noexcept;

scale_type scale() const noexcept;
scale_type current_scale() const noexcept;
bool isfloat() const noexcept;

void set_user_scale(scale_type) noexcept;
void adjust_scale_by(scale_type) noexcept;
void change_scale_to(scale_type) noexcept;

//
// Observers:
//
bool iszero() const noexcept;
bool isneg() const noexcept;
bool ispos() const noexcept;
bool isnan() const noexcept;
bool isinf() const noexcept;
bool isinf(bool negative) const noexcept;

int signum() const;

decimal rawval() const;

//
// Mutators:
//
void clear() noexcept;
void all_clear() noexcept;

decimal& set_to_zero() noexcept;
decimal& set_to_one(bool negative = false);

decimal& negate() noexcept;
decimal& abs() noexcept;

decimal& round();
decimal& round(int number_of_digits);
decimal& round(int number_of_digits, rounding);

//
// Comparisons:
//
std::partial_ordering operator<=>(const decimal&) const;
bool operator==(const decimal&) const;

//
// Arithmetic:
//
decimal& operator++();
decimal& operator--();

decimal operator++(int);
decimal operator--(int);

decimal& operator+=(const decimal&);
decimal& operator-=(const decimal&);
decimal& operator*=(const decimal&);
decimal& operator/=(const decimal&);

//

```

```

// Size, etc.:
//
std::size_t digits();
std::size_t unnormalized_digits() const noexcept;

bool empty() const noexcept;
std::size_t size() const noexcept;
void shrink_to_fit();
};

//
// Miscellaneous non-member functions:
//
using std::swap;
void swap(decimal&, decimal&);

integer to_integer(const decimal&, rounding = rounding::all_to_zero) const;

std::string to_string(const decimal&, bool scientific = false);
std::string to_string(const decimal&, int precision, bool scientific = false);

//
// Non-member operators:
//
decimal operator+(const decimal&);
decimal operator-(const decimal&);

decimal operator+(const decimal&, const decimal&);
decimal operator-(const decimal&, const decimal&);
decimal operator*(const decimal&, const decimal&);
decimal operator/(const decimal&, const decimal&);

//
// <cmath>-like functions:
//
decimal abs(const decimal&) noexcept;
decimal ceil(const decimal&);
decimal floor(const decimal&);
decimal trunc(const decimal&);
decimal round(const decimal&, rounding = rounding::tie_away_zero);
decimal rint(const decimal&);
decimal modf(const decimal&, decimal*);
decimal fmod(const decimal&, const decimal&);
decimal remainder(const decimal&, const decimal&, rounding = rounding::tie_to_even);
decimal sqr(const decimal&);
decimal copysign(const decimal&, const decimal&);
decimal fma(const decimal&, const decimal&, const decimal&);

```

Detailed descriptions

The decimal Class

A Member Type and Some Constants

```
using scale_type = int;
```

A type alias for the scale type might be useful for documentation or for making code clear about what's being passed to a function. This should probably be a machine word for efficiency.

```
static constexpr scale_type max_scale = implementation-defined;
```

The implementation may limit the values passed to `scale_type` function arguments to $\pm \text{max_scale}$. This should be at least `max_digits` below.

```
static constexpr scale_type floating_scale = INT_MIN;
```

The class normally acts like a fixed point type (at least when the value becomes externally visible), but you can make it act like a floating point type by explicitly setting the desired scale to `floating_scale`. This just turns off rounding except for division.

```
static constexpr int max_digits = implementation-defined;
```

As a practical matter, we can't have an infinite number of digits; so when using this class as a fixed point type, exposing the value to the outside world will round the value to at most `max_digits` digits. This can also happen if the class is being used as a floating point type when division would generate more than `max_digits` digits, in which case division will generate a guard digit and then round.

The class should be able to represent at least 38 decimal digits.

One use for this class could be in a database access library, and the author notes that both Oracle and SQL Server have exact numeric types that can hold up to 38 decimal digits. That's where the 38 comes from. (ISO/IEC 9075, the SQL standard, doesn't specify any value for the precision, and Postgresql allows up to 131072 digits; but that seems over the top.)

Note that we don't call this a `scale_type` because it's a precision, not a scale.

```
static constexpr rounding default_rounding = rounding::tie_to_even;
```

If you don't specify a rounding mode when calling a function that takes a rounding argument, that argument will default to `default_rounding` except as noted.

```
static constexpr rounding bankers_rounding = rounding::tie_away_zero;
```

As a convenience for users, there's also a symbol for a rounding mode that might be more appropriate for financial applications, but this library makes no use of it.

```
static constexpr decimal inf = a value that represents positive infinity;  
static constexpr decimal nan = a value that represents a quiet NaN;
```

We should probably also have symbols for infinities and NaNs.

One likely implementation would have decimal digits stored in a sequence container with random-access iterators, in which case we might not be able to say `constexpr` unless the container has a `constexpr` default constructor. That's only vector in [N4917](#). Should we allow the implementation to be a deque?

Special Member Functions and swap

```
explicit decimal(rounding = default_rounding, scale_type = 0);
```

The default constructor constructs a decimal equal to zero; and it can optionally be used to initialize the rounding mode and the user's requested scale.

```
decimal(const decimal&);  
decimal& operator=(const decimal&);
```

```
decimal(decimal&&);  
decimal& operator=(decimal&&);
```

```
~decimal() noexcept;
```

```
void swap(decimal&);
```

decimals are freely copyable, moveable, and swappable.

Construction from Other Types

```
template<typename FundamentalArithmeticType>
decimal(FundamentalArithmeticType, scale_type = ?, rounding = default_rounding);
```

An implementation would probably provide three constructor templates for implicit conversion from fundamental arithmetic types, one for signed integers, one for unsigned integers, and one for floating point values. If the only argument is an integer, the scale defaults to zero, otherwise it defaults to `floating_scale`.

```
explicit decimal(const integer&, scale_type = 0, rounding = default_rounding);
```

There's no implicit conversion from integers because both `decimal` and `integer` have implicit conversions from fundamental arithmetic types which could lead to ambiguity.

```
explicit decimal(const char* value, rounding = default_rounding, const char** term_ptr = nullptr);
explicit decimal(const std::string& value, rounding = default_rounding, std::size_t* term_pos = nullptr);
```

Instances can also be explicitly constructed from C-style strings and `std::strings`. When constructing from strings, the scale is inferred from the position of the decimal point (or its absence). The string may begin with a '+' or a '-', and it may be in either fixed-point or scientific notation. The 'e' in scientific notation may be upper or lower case.

An optional third argument can be used to discover the character that terminated the parse:

- If `term_ptr` is not `nullptr`, `*term_ptr` will be set to a pointer to the character that terminated the parse, which will be a pointer to a '\0' if the parse continued to the end of the string.
- If `term_pos` is not `nullptr`, `*term_pos` will be set to the position of the character that terminated the parse, which will be `value.size()` if the parse continued to the end of the string.

Issue: should construction from strings be templates? We expect that we'll be dealing with only '+', '-', '.', 'E', 'e' and decimal digits, but the number could be part of a larger string in some language other than English. Also, should we allow non-Arabic (e.g., Devanagari) digits? This issue also applies to assignment from strings below.

Assignment from Other Types

```
template<typename FundamentalArithmeticType>
decimal& operator=(FundamentalArithmeticType);
```

Instances can be assigned values of any fundamental arithmetic type. The desired scale and rounding mode will default as do the implicit constructors.

```
template<typename FundamentalArithmeticType>
decimal& assign(FundamentalArithmeticType, scale_type = ?, rounding = default_rounding);
```

There's also an `assign` member template that allows you to specify a scale and a rounding mode. As with the implicit constructors, if the only argument is an integer, the scale will default to zero; but if it's a floating point value, the scale will default to `floating_scale`.

```
decimal& assign(const integer&, scale_type = 0, rounding = default_rounding);
```

You can also explicitly assign integers and optionally specify a scale and rounding mode. There's no `operator=(integer)` because both `decimal` and `integer` have implicit conversions from fundamental arithmetic types which could lead to ambiguity.

```
decimal& assign(const char*, rounding = default_rounding, const char** = nullptr);
decimal& assign(const std::string&, rounding = default_rounding, std::size_t* = nullptr);
```

Assignment from strings behaves like [construction from strings](#).

Conversion to Other Types

```
explicit operator bool() const noexcept;
```

The conversion to `bool` returns whether `*this` is non-zero. It's intended to support the “`if(my_decimal)`” idiom. It returns `false` if `*this` is a NaN.

```
explicit operator long double() const noexcept;
```

Conversion to `long double` is also provided. If `*this` is finite but outside the range, `[LDBL_MIN,LDBL_MAX]`, the function will return `HUGE_VALL` with `errno` set to `ERANGE`. Conversion to `long double` can quietly result in loss of precision.

Issue: should we throw an exception on overflow instead of returning `HUGE_VALL`? Or maybe return infinity even if `*this` is finite but too big?

Issue: should we have explicit conversions to *all* the fundamental arithmetic types? This could allow finer detection of overflow if we throw exceptions.

Issue: what behavior should be specified if the `decimal` class supports NaNs and/or infinities but the C++ implementation's `long double` doesn't?

```
integer to_integer(rounding = rounding::all_to_zero) const;
```

The `to_integer` function makes a copy of `*this` and then rounds the copy. By default, the value will be truncated toward zero; but you can specify a different rounding mode if you need to. This function will throw an exception if `*this` is a NaN or an infinity.

```
std::string to_string(bool scientific = false) const;
std::string to_string(int precision, bool scientific = false) const;
```

The `to_string()` function makes a copy of `*this`, rounds the copy, and then returns that value in either fixed-point or scientific notation. A decimal point will be a period. The string can begin with `'-'`, but it will never begin with `'+'`.

If `precision` is not passed, it will default to a number of digits appropriate given the user's requested scale; otherwise, the function rounds to `precision` decimal digits, or to [max_digits](#) digits if `precision` is greater than `max_digits`. If `precision` is exactly `INT_MIN`, no rounding will occur.

In fixed-point notation (`scientific == false`):

- If the value is an integer, there will be no decimal point.
- If the absolute value is less than one, the string (after a possible `'-'`) will begin with `"0."`; and there could be insignificant zeros after the decimal point.
- If the absolute value requires more digits than are currently stored, there could be trailing zeros.

In scientific notation (`scientific == true`):

- If only one digit is produced, there will be no decimal point.
- The `'e'` will always be lower case.
- The exponent will be returned as at least two digits, possibly with a leading zero.

The `scientific` and `precision` arguments are intended mainly to generate strings that are suitable for the output stream `<<` operator, but there's no compelling reason to keep them secret and disallow their use for other purposes.

Rounding and Scale

```
rounding get_rounding() const noexcept;
void set_rounding(rounding = default_rounding) noexcept;
```

The user's requested rounding mode can be examined and changed after construction.

```
int scale() const noexcept;
int current_scale() const noexcept;
```

```
bool isfloat() const noexcept { /*as if*/ return scale() == floating_scale; }
```

`scale()` returns the user's desired scale; `current_scale()` returns the scale of the current representation before rounding.

```
void set_user_scale(scale_type) noexcept;  
void adjust_scale_by(scale_type) noexcept;  
void change_scale_to(scale_type) noexcept;
```

`set_user_scale(scale_type)` allows the user to set the desired scale after construction without changing the represented value.

`adjust_scale_by(scale_type adj)` adds *adj* to the current internal scale without changing the raw (unscaled) value. This has the effect of multiplying the represented value by 10^{-adj} .

`change_scale_to(scale_type new_scale)` just sets the current internal scale. This has the effect of arbitrarily setting the represented value to `rawval(.)` $\times 10^{-new_scale}$.

All these functions run in constant time.

Observers

```
bool iszero() const noexcept;  
bool isneg() const noexcept;  
bool ispos() const noexcept;  
bool isnan() const noexcept;  
bool isinf() const noexcept;  
bool isinf(bool negative) const noexcept;
```

All except `isnan()` return false if `*this` is a NaN. All run in constant time.

```
int signum() const;
```

`signum()` returns +1 if `*this` is greater than zero, 0 if `*this` is equal to zero, or -1 if `*this` is less than zero. It throws an exception if `*this` is a NaN, otherwise it runs in constant time.

```
decimal rawval() const;
```

`rawval()` just returns a copy of `*this` with the current scale set to zero, the effect being to return an integer value made of the currently represented digits. It doesn't change the user's requested scale.

Mutators

```
void clear() noexcept;  
void all_clear() noexcept;  
decimal& set_to_zero() noexcept;
```

`clear()` and `set_to_zero()` both assign zero to `*this` without changing the user's requested scale or rounding mode. `all_clear()` has the additional effect of setting the requested scale to zero and the rounding mode to `default_rounding`.

```
decimal& set_to_one(bool negative = false);
```

`set_to_one(false)` assigns +1 to `*this`; `set_to_one(true)` assigns -1 to `*this`.

```
decimal& negate() noexcept;
```

`negate()` changes the sign of `*this` if `*this` is non-zero and not a NaN. It will never create a negative zero (or a negative NaN, whatever that could mean). It runs in constant time.

```
decimal& abs() noexcept;
```


`abs()` unconditionally makes `*this` non-negative. It runs in constant time.

```
decimal& round();  
decimal& round(int number_of_digits);  
decimal& round(int number_of_digits, rounding);
```

The `round()` function with no argument does nothing if the user requested [floating_scale](#), otherwise it rounds to a number of digits appropriate given the user's requested scale using the user's requested rounding mode.

The other overloads round to a particular number of decimal digits without regard to the user's requested scale (and so round even if the user requested `floating_scale`). If no rounding mode is specified, it defaults to the user's requested mode. If `number_of_digits` is less than one, `*this` is set to zero.

All but `clear()` and `all_clear()` return `*this`.

Comparisons

```
std::partial_ordering operator<=>(const decimal&) const;  
bool operator==(const decimal&) const;
```

If it's decided that decimals needn't support NaNs, then the spaceship operator could return `strong_ordering`.

Arithmetic

```
decimal& operator++();  
decimal& operator--();  
  
decimal operator++(int);  
decimal operator--(int);
```

The increment and decrement operators add or subtract 1.0 as expected; they don't just add ± 1 to the currently represented LSD.

```
decimal& operator+=(const decimal&);  
decimal& operator-=(const decimal&);  
decimal& operator*=(const decimal&);  
decimal& operator/=(const decimal&);
```

Any operation for which either operand is a NaN yields a NaN; otherwise addition, subtraction and multiplication are exact and so can generate more than [max_digits](#) digits. Division, if it would generate a quotient of more than `max_digits` digits, generates a guard digit and then rounds using the user's requested rounding mode.

Zero divided by zero and infinity minus infinity yield a NaN; other division by zero yields $\pm$ infinity.

Size, etc.

```
std::size_t digits();  
std::size_t unnormalized_digits() const noexcept;
```

`unnormalized_digits()` returns the number of decimal digits currently represented (before rounding).

`digits()` first calls `this->round()` and then returns the number of digits represented. This could be fewer than the number of significant digits if trailing zeros to the right of the decimal point are not stored.

```
bool empty() const noexcept;  
std::size_t size() const noexcept;  
void shrink_to_fit();
```

The internal representation of the value is probably stored in a standard container of some sort; and `empty()`, `size()` and `shrink_to_fit()` all just call that container's functions of the same name.

Non-member Functions

Miscellaneous Functions

```
using std::swap;
void swap(decimal&, decimal&);

integer to_integer(const decimal&, rounding = rounding::all_to_zero) const;

std::string to_string(const decimal&, bool scientific = false);
std::string to_string(const decimal&, int precision, bool scientific = false);
```

All have the same semantics as do their corresponding member functions.

Operators

```
decimal operator+(const decimal&);
decimal operator-(const decimal&);
```

The unary `+` operator returns a copy of its argument; the unary `-` operator returns a negated copy of its argument.

```
decimal operator+(const decimal& lhs, const decimal& rhs);
decimal operator-(const decimal& lhs, const decimal& rhs);
decimal operator*(const decimal& lhs, const decimal& rhs);
decimal operator/(const decimal& lhs, const decimal& rhs);
```

Any operation for which either operand is a NaN yields a NaN; otherwise addition, subtraction and multiplication are exact and so can generate more than [max_digits](#) digits. Division, if it would generate a quotient of more than `max_digits` digits, generates a guard digit and then rounds using the user's requested rounding mode.

Zero divided by zero and infinity minus infinity yield a NaN; other division by zero yields $\pm$ infinity.

The result will have *lhs*'s desired scale and rounding mode. Note that this implies that addition and multiplication are not commutative in the strictest sense of that term, although they will be commutative as far as the value of the sum or product is concerned.

<cmath>-like Functions

```
decimal abs(const decimal&) noexcept;
decimal ceil(const decimal&);
decimal floor(const decimal&);
decimal trunc(const decimal&);
decimal round(const decimal&, rounding = rounding::tie_away_zero);
decimal rint(const decimal&);
decimal modf(const decimal&, decimal*);
decimal fmod(const decimal&, const decimal&);
decimal remainder(const decimal&, const decimal&, rounding = rounding::tie_to_even);
decimal sqr(const decimal&);
decimal copysign(const decimal&, const decimal&);
decimal fma(const decimal&, const decimal&, const decimal&);
```

A few <cmath>-like functions that are expected to return exact values are supplied. All have behavior that mimics the standard functions of the same name.

Note that the non-member `round` function allows you to specify a rounding mode. This defaults to `tie_away_zero` which mimics `std::round()`.

The remainder function also allows specifying a rounding mode for the quotient of the first two arguments. This defaults to `tie_to_even` which yields the IEEE 754 remainder (like `std::remainder()` does); but users can compute other kinds of remainders if they want to.

Issue: should we also require `sqrt()` even though it likely wouldn't return an exact value? If so, should it take an optional rounding argument?

The `fma` function doesn't do anything special since no rounding would occur between the multiplication and the addition in any event; but the function is included because it's "canonical", and because it might simplify updating legacy code to use decimals. The `FP_FAST_FMA` macro tells you nothing about this function.

For all of these functions, the result's requested scale and rounding mode will be the same as those of the first, or only, argument.
